

CS327 course plan

(Jan-May 2026) (class strength: 82)

Instructor:

[Shouvick Mondal](#) (*doubt sessions*: by appointment ONLY at AB13/402A)

*For each student, there will be TWO lectures, and ONE lab session per week. The L-T-P-C for this course is (3-0-1.5-5) which means a weekly load of: 3 hrs of lecture, and 1.5 hrs of lab, and overall 5 credits for this course. **Prerequisites:** (i) Data Structures and Algorithms-I (ES 242), (ii) Basics of Regular Languages and Context Free Languages.*

TAs:

Devansh Yadav, Madhav Raj Verma

Course email (all announcements will be made): cs327_compilers-jan-may-2026.pvtgroup@iitgn.ac.in

Google classroom (all lab assignments will be posted): <https://classroom.google.com/c/ODMzMzQ4Mjk3MzQ5?cjc=kfbqyhwv>

GitHub: <https://github.com/SET-IITGN/CS-327-Compilers>

Lectures:

- (Slot **F1+G2**): Mon+Fri, 11:30 AM – 12:50 PM @ AB1/101 (Surendra C Sheth Learning Theatre)

Lab sessions:

- (Slot **F2**): Thu, 11:30 AM – 12:50 PM
 - Batch 1.1 (@ AB10/203)
 - Batch 1.2 (@ AB10/204)

Evaluation *(relative grading)*:

- ✓ [A1: 7 marks] Lab Assignment #1 [Deadline: Jan 24]
- ✓ [A2: 13 marks] Lab Assignment #2 [Deadline: Feb 20]
- ✓ [E1: 20 marks] Exam I [Feb 23, 2:00–4:00 PM, 10/103]
- ✓ [A3: 20 marks] Lab Assignment #3 [Deadline: Apr 03]
- ✓ [A4: 20 marks] Lab Assignment #4 [Deadline: Apr 18]
- ✓ [E2: 15 marks] Exam II [Apr 27, 2:00–4:30 PM, 10/103]
- ✓ [B: 5 marks] Attendance (TAs must record attendance in physical signature for both lectures and lab. sessions)

There will be no makeup exams, re-exams, or reschedules.

There will be no further changes to the lecture/lab slots.

All deadlines are firm, set at 23:59:59 hrs IST. Deadlines for TAs are +5 days from students' deadline.

Lab Assignments (A2, A3, A4) will be linked, i.e., built upon the previous one, and should be performed in groups of size at most four. Each group will develop a compiler for a custom language of their choice, or a subset of any popular programming language. Compilation phases collectively implemented (spanning across all four lab assignments) need not be exhaustive.

Software Tools to be used: [lex](#), [yacc](#), [gcc](#), [gas](#), and other relevant tools as and when necessary.

Overall course load:

{23L (x1)} + {12P (x1)}

Blue (lec.): AB1/101 (150x)

Red (lab.): AB10/203 (70x), AB10/204 (70x)

Timeline:

Excluding holidays, exam weeks, and breaks. Striked out classes to be rescheduled on mutual convenience.

Month	Day	Topics (slides)
January	5, 12, 16, 19, 23, 30 {8, 15} (A1), {22, 29} (Lab Assignment A1): • Introduction to Language Translation and Tokenization • Part 1: Understanding the Compilation Pipeline • Part 2: Compiler Optimization • Part 3: Tokenizer Implementation in C • This is an INDIVIDUAL assignment (Lab Assignment A2): • This is a GROUP assignment for building a Lexical Analyzer • See also: A Guide to lex & yacc • See also: lex & yacc, 2nd Edition • Hands-on session delivered on 22/01/2026: > Recap of basic concepts in Lexical Analysis discussed in lectures 3 and 4. > lex implementation to specify regular expressions for strings over the alphabet $\Sigma=\{a, b\}$ that: (i) start and end with a, (ii) have the third last letter as a, (iii) have exactly three bs, and (iv) have even length. > lex implementation to count the number of he and she in a text. Also, count all possible substrings he → use of REJECT > C++ implementation of KMP and demonstration of failure $f()$ for: (i) ababaa, (ii) abaababa (fibonacci string $s_6=s_5s_4$). • Recap of lex with yacc on 29/01/2026 > See also: Python3 lexical structure > See also: Python3 grammar	(Lecture 1): • Course logistics • Introduction to compilers, and other language translators • Introduction to compilation phases of a typical compiler • https://github.com/SET-IITGN/CS-327-Compilers (appendix) (Lecture 2): • Static versus Dynamic scoping • Parameter passing, and translation discrepancies • Examples of lex and yacc for building custom language processors • https://github.com/SET-IITGN/CS-327-Compilers (walkthrough) • Recommended reading+exercises: Chapter 1 of Dragon Book (Lecture 3): • Lexing (Rupesh Nasre's Slides 1–26) • Basic Lex, Input Buffering • https://github.com/SET-IITGN/CS-327-Compilers/tree/main/lec3ex1 (src/bin) • https://github.com/SET-IITGN/CS-327-Compilers/tree/main/lec3ex2 (src/bin) • Recommended reading+exercises: Chapter 3 of Dragon Book (Lecture 4): • Lexing (Rupesh Nasre's Slides 27–73) • Knuth–Morris–Pratt (KMP) String Matching algorithm • KMP visualization (from Yves Lucet) • https://github.com/SET-IITGN/CS-327-Compilers/tree/main/lec4 (src/bin) • Regex → NFA → DFA (via McNaughton–Yamada–Thompson algorithm) • Regex → DFA (directly as used in lex) • Recommended reading+exercises: Chapter 3 of Dragon Book (Lecture 5): • Interfacing lex with yacc. • yacc specification of a Context-free grammar (CFG) where the precedence of addition and multiplication are swapped. This means + has higher precedence than *. <pre> E -> E * T T T -> T + F F F -> NUM </pre> Arbitrary length via lexer [0-9]+ → single NUM token. https://github.com/SET-IITGN/CS-327-Compilers/tree/main/mul_add_pred_swap (src/bin) • The way forward: overview of Syntax Directed Translation (SDT), attribute grammar, and semantic actions. • ANSI C grammar, lex specification (2011 ISO C standard) • ANSI C yacc grammar (2011 ISO C standard) • Scanning & Parsing for yap1: a C-like Language • See also: Scanning & Parsing for pro: well-formed formula (wff) in Propositional Logic. Sample input: “$\sim(P \rightarrow Q) \wedge ((Q \wedge P) \rightarrow Q)$”, “$(P \wedge Q)$” (Lecture 6): • Introduction to Parsing (Fredrik Kjolstad's Slides 1–51) • Context Free Grammars (CFGs), Derivations and Parse Trees, Ambiguity

February

1, 2, 9, 13, 16
5, 12} (A2), {19

(Lab Assignment A2) contd.:

- This is a GROUP assignment for building a Lexical Analyzer
- Review of draft language specifications done on 05/02/2026
- Regular lab. on 12/02/2026
- Regular lab. on 19/02/2026

Exam I (midsem)

(Lecture 7):

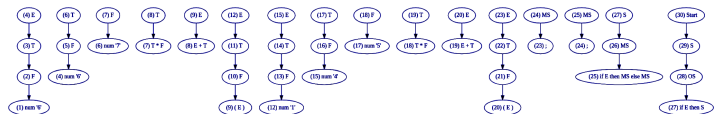
- Introduction to Parsing (Fredrik Kjolstad's Slides 52–57)
- Context-Free Grammars (Keith Schwarz's Slides 1–32)
- (lex and yacc) Implementation of Ambiguous dangling-else Grammar

stmt -> if expr then stmt
| if expr then stmt else stmt
| other



- (lex and yacc) Implementation of Unambiguous dangling-else Grammar

stmt -> matched_stmt
| open_stmt
matched_stmt -> if expr then matched_stmt else matched_stmt
| other
open_stmt -> if expr then stmt
| if expr then matched_stmt else open_stmt



(Lecture 8):

- Context-Free Grammars (Keith Schwarz's Slides 33–45)
- Parsing (Rupesh Nasre's Slides)
- See also: The TIOBE Index
- See also: The Go Programming Language Specification
- See also: The Scala Programming Language Specification
- See also: TabError behavior doesn't match documentation
<https://bugs.python.org/issue24260>

(Lecture 9):

- Parsing (Rupesh Nasre's Slides 14–37)

(Lecture 10):

- Parsing (Rupesh Nasre's Slides 37–49)
- Discussion on Recursive-Descent, and Predictive Parsing
- Walkthrough of a manual construction of FIRST and FOLLOW sets for:

E -> T E'
E' -> + T E' | ε
T -> F T'
T' -> * F T' | ε
F -> (E) | id

- LL(1) Parsing Table Builder (from Zak Kincaid and Shaowei Zhu)
- Recommended reading+exercises: Chapter 4 of Dragon Book
- See also: (lex and yacc) Infix to Postfix Translation (uses %union)

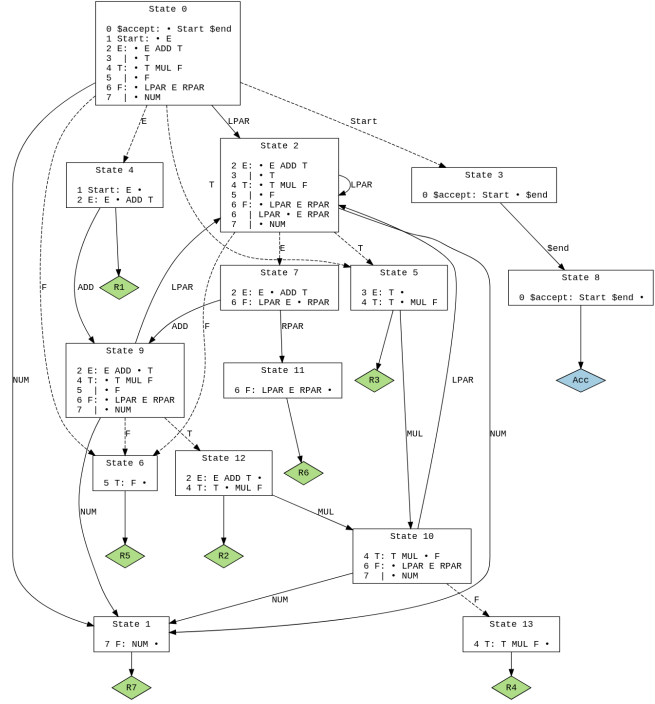
Input (source code)	Syntax Directed Translation (target code)
2 + 2	22+
a+2/6+7*5	a26/+75*+
a+(a+(a+(a+1)))	aaaa1++++
aa+bb+ab+ab	aabb+ab+ab+
A*(B+D)/E-F*(G+H/K)	ABD+*E/FGHK/+*-

March

9, 13, 16, 20, 23, 27, 30
11, 19, 26} (A3)

(Lab Assignment A3):

- This is a GROUP assignment on extending A2 to Syntax Analyzer and implementing A2 specifications.
- Hands-on session delivered on 11/03/2026:
 - Bottom-Up parsing: [LR parsing and LR\(0\) Automaton](#)



- Regular lab. on 19/03/2026
- Regular lab. on 26/03/2026

(Lecture 11):

- Recap of (lex and yacc) Infix to Postfix Translation (uses %union)
- Parsing (Rupesh Nasre's [Slides 45–57](#))
- Recommended reading+exercises: Chapter 4 of Dragon Book

(Lecture 12):

- Brief recap of FIRST and FOLLOW sets for LL(1) parsing
- Example of Panic Mode Error Recovery in LL(1) parsing
- Parsing (Rupesh Nasre's [Slides 58–81](#))
- Recommended reading+exercises: Chapter 4 of Dragon Book

LL(1) grammar	Non-terminal	FIRST	FOLLOW
$E \rightarrow T E'$	E	{(, id}	{), \$}
$E' \rightarrow * T E'$	E'	{*, /, ε}	{), \$}
$E' \rightarrow / T E'$			
$E' \rightarrow ε$			
$T \rightarrow F T'$	T	{(, id}	{*, /,), \$}
$T' \rightarrow + F T'$			
$T' \rightarrow - F T'$			
$T' \rightarrow ε$	T'	{+, -, ε}	{*, /,), \$}
$F \rightarrow (E) id$	F	{(, id}	{*, /, +, -, \$}
$F \rightarrow id (E)$			

	*	/	+	-	(	)	id	\$
E	skip	skip	skip	skip	$E \rightarrow T E'$	sync	$E \rightarrow T E'$	sync
E'	$E' \rightarrow * T E'$	$E' \rightarrow / T E'$	skip	skip	skip	$E' \rightarrow ε$	skip	$E' \rightarrow ε$
T	sync	sync	skip	skip	$T \rightarrow F T'$	sync	$T \rightarrow F T'$	sync
T'	$T' \rightarrow + F T'$	$T' \rightarrow - F T'$	skip	skip	skip	$T' \rightarrow ε$	skip	$T' \rightarrow ε$
F	sync	sync	sync	sync	$F \rightarrow (E) id$	sync	$F \rightarrow id (E)$	sync

Stack (->)	Input (->)	message
E\$	id+id\$	
TE\$	id+id\$	
FT\$	id+id\$	
id(E)TE\$	id+id\$ match id	
(E)TE\$	id\$Error, exp (got +, pop (	
E)TE\$	id\$Skip +	
E)TE\$	id\$	
TE)TE\$	id\$	
FT)TE\$	id\$	
id(E)TE)TE\$	id\$ match id	
(E)TE)TE\$	\$Error, exp (got \$, pop (	
E)TE)TE\$	\$Error, sync: pop E	
)TE)TE\$	\$Error, exp) got \$, pop)	
TE)TE\$	\$Error, sync: pop T'	
E)TE\$	\$Error, sync: pop E'	
)TE\$	\$Error, exp) got \$, pop)	
TE\$	\$Error, sync: pop T'	
E\$	\$Error, sync: pop E'	
\$	\$ parsing complete	

Stack (->)	Input (->)	message
E\$	id+id\$	
TE\$	id+id\$	
FT\$	id+id\$	
id(E)TE\$	id+id\$ match id	
(E)TE\$	id\$Error, exp (got +, pop (	
E)TE\$	id\$Skip +	
E)TE\$	id\$	
TE)TE\$	id\$	
FT)TE\$	id\$	
id(E)TE)TE\$	id\$ match id	
(E)TE)TE\$	\$Error, exp (got \$, pop (	
E)TE)TE\$	\$Error, sync: pop E	
)TE)TE\$	\$Error, exp) got \$, pop)	
TE)TE\$	\$Error, sync: pop T'	
E)TE\$	\$Error, sync: pop E'	
)TE\$	\$Error, exp) got \$, pop)	
TE\$	\$Error, sync: pop T'	
E\$	\$Error, sync: pop E'	
\$	\$ parsing complete	

(Lecture 13):

- Recap of LR(0) Automaton, SLR(1) Parsing Table
- Parsing (Rupesh Nasre's [Slides 82–108](#))
- LR(0) Automaton, and Shift-Reduce Parsing – Viable Prefixes
- LR(1) Automaton, and CLR(1) Parsing Table
- [LR\(1\) Itemset Generator](#)
- [LR\(1\) Automaton + CLR\(1\) Parsing Table \(10 states\)](#) for:
 - $S' \rightarrow S$
 - $S \rightarrow C C C$
 - $C \rightarrow c c c | d$
- Automaton associated with LALR(1) parsing table (7 states) for the above
- lex and yacc implementations for examples in slides: [ex1](#), [ex2](#), [ex3](#), [ex4](#)
- Recommended reading+exercises: Chapter 4 of Dragon Book
- See also: [Understanding and resolving conflicts in yacc](#)

(Lecture 14):

- Parsing (Rupesh Nasre's [Slides 97–108](#))
- Recap of LR(1) Automaton, CLR(1) Parsing Table, LALR(1) Parsing Table
- State Merging in LALR may produce conflict:

$S' \rightarrow S$
 $S \rightarrow a A d | b B d | a B e | b A e$
 $A \rightarrow c$

B -> c

- Introduction to Syntax Directed Translation (Rupesh Nasre's [Slides 1-9](#))
- Recap of ([lex and yacc](#)) Infix to Postfix Translation (uses [%union](#))
- Recommended reading+exercises: Chapters 2, 5 of Dragon Book
- See also: [Understanding and resolving conflicts in yacc](#)
- See also: [Expressive Diagnostics](#) in [clang](#)
- See also: [Miscellaneous Parsing](#) (from Maggie Johnson and Julie Zelenski)

(Lecture 15):

- Recap + Syntax Directed Translation (Rupesh Nasre's [Slides 1-23](#))
- Syntax Directed Definition (SDD), and Syntax Directed Translation (SDT)
- Synthesized/ Inherited Attributes, Order of Evaluation, Dependency Graph
- Annotated Parse Tree, S-attributed/ L-attributed/ non L-attributed SDD
- SDT Applications: Creating an explicit Syntax Tree
- Recommended reading+exercises: Chapter 5 of Dragon Book
- See also: [Attribute Grammar](#)

(Lecture 16):

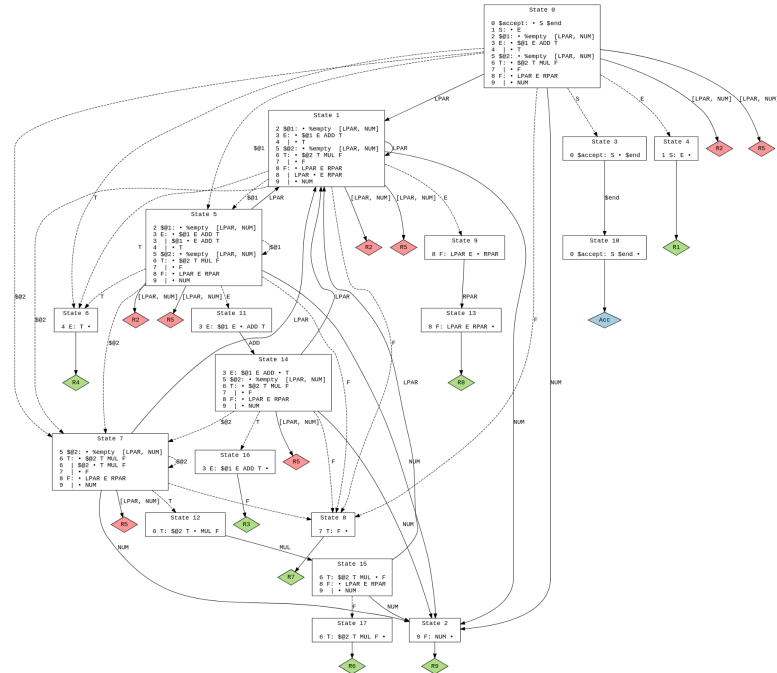
- Syntax Directed Translation (Rupesh Nasre's [Slides 22-27](#))
- Binary to Decimal conversion using Semantic Rules

Grammar	S-attributed SDD with Semantic Rules
$S \rightarrow L_1 . L_2$	$S \rightarrow L_1 . L_2 \{S.v = L_1.v + (L_2.v / \text{pow}(2, L_2.s));\}$
$S \rightarrow L$	$S \rightarrow L \{S.v = L.v;\}$
$L_1 \rightarrow L_2 B$	$L_1 \rightarrow L_2 B \{L_1.v = 2 * L_2.v + B.v; L_1.s = L_2.s + 1;\}$
$L \rightarrow B$	$L \rightarrow B \{L.v = B.v; L.s = 1;\}$
$B \rightarrow 0$	$B \rightarrow 0 \{B.v = 0;\}$
$B \rightarrow 1$	$B \rightarrow 1 \{B.v = 1;\}$

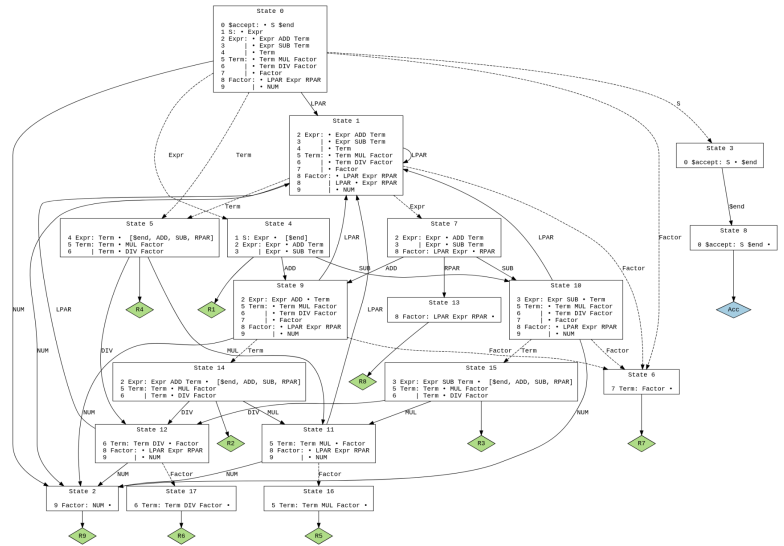
- Recommended reading+exercises: Chapter 5 of Dragon Book
- See also: [Semantics of Context-Free Languages](#)

(Lecture 17):

- Syntax Directed Translation (Rupesh Nasre's [Slides 26-36](#))
- [Infix to Prefix Translation \(representation only\) w/ CONFLICTS](#)



- [Infix to Prefix Translation \(representation only\) w/o CONFLICTS](#)



- Recommended reading+exercises: Chapter 5 of Dragon Book

(Lecture 18):

- Intermediate Code Generation (Rupesh Nasre's [Slides 1–10](#))
- Recommended reading+exercises: Chapter 6 of Dragon Book
- See also: [Intermediate Representation \(IR\)](#)
- See also: [LLVM Intermediate Representation](#)

April

9, 10, 13, 17, 20
{2, 16} (A4)

(Lab Assignment A3):

- Regular lab. on 02/04/2026

(Lab Assignment A4):

- This is a GROUP assignment on extending A3 to Intermediate Code Generation and implementing all pending specifications in previous assignments.
- Regular lab. on 16/04/2026

Exam II (endsem)

(Lecture 19):

- Recap + Intermediate Code Generation (Rupesh Nasre's [Slides 11–20](#))
- Recommended reading+exercises: Chapter 6 of Dragon Book
- See also: [GCC Internals - Liveness Information](#)
- See also: [Static Single Assignment \(SSA\) form](#), [Type System](#)

(Lecture 20):

- Intermediate Code Generation (Rupesh Nasre's [Slides 18–31](#))
- Recommended reading+exercises: Chapter 6 of Dragon Book

(Lecture 21):

- Intermediate Code Generation (Rupesh Nasre's [Slides 32–51](#))
- Recommended reading+exercises: Chapter 6 of Dragon Book

(Lecture 22):

- Code Generation (Rupesh Nasre's [Slides 1–23](#))
- Recommended reading+exercises: Chapter 8 of Dragon Book

(Lecture 23):

- Code Generation (Rupesh Nasre's [Slides 23–44](#))
- Recommended reading+exercises: Chapter 8 of Dragon Book

Course Contents:

(tentative and subject to change at the sole discretion of the instructor)

1. Overview of Compiler; Brief discussion on Assembler, Linker, Loader
2. Lexical Analysis; Syntax Analysis: Parsers, Top-down, Bottom-up, LR Parsers, Parser generators;
3. Syntax directed translation; Intermediate code generation and type checking; Run-time Environments; Type checking;
4. Code generation and optimization (data flow analysis, instruction optimization, loop optimization, memory optimization etc).
5. Tools: Flex, Bison (or similar software).

Student Honour Code: <https://iitgn.ac.in/students/honourcode/>

Texts/References:

1. Alfred Aho, Monica Lam, Ravi Sethi, Jeff Ullman. [*Compilers: Principles, Techniques, and Tools*](#) (Dragon Book), Pearson Education India; 2nd Edition, 2013.
2. Steven Muchnick, [*Advanced Compiler Design and Implementation*](#), Morgan Kaufmann; 1st edition 1997.
3. John R. Levine, Tony Mason, and Doug Brown, [*Lex & Yacc*](#) (2nd ed.). O'Reilly & Associates, Inc., USA, 1992.
4. Jonathan Bartlett, [*Programming from the Ground Up*](#), 2003.
5. Wikibooks, [*x86 Assembly*](#), [*GNU assembly syntax*](#) (online).

Learning Outcomes:

Students should understand the basics of programming language design, as well as the execution environment. Students should be able to build a compiler for a subset of a realistic programming language (C/C++/Java), implementing the basic building blocks of a compiler.

Other Remarks:

This course can be credited as a CSE minor course.